

Case Study: Employee Salary Calculation Using Callable and Future

Scenario

A company has an employee management application that calculates an employee's monthly salary.

The salary calculation includes:

- Basic salary
- Allowances
- Performance bonus
- Tax deduction
- Other deductions

The calculation may take some time because the application needs to retrieve attendance, performance, and tax details from different systems.

Problem

If the salary calculation is performed directly by the main application thread, the application may become unresponsive until the calculation is completed.

The company wants the salary calculation to execute in a separate thread and return the calculated salary when it is ready.

Solution Using Callable

The salary calculation is represented as a `Callable` task because the task must return the final salary amount.

The `Callable` task performs the following activities:

1. Retrieves the employee's attendance details.
2. Retrieves performance bonus information.
3. Calculates allowances.
4. Calculates tax and other deductions.
5. Returns the final net salary.

Role of ExecutorService

The `ExecutorService` manages the worker thread that performs the salary calculation.

The main application submits the salary calculation task to the executor instead of executing it directly.

Role of Future

When the task is submitted, the application immediately receives a `Future` object.

The `Future` represents the salary result that will become available after the calculation is completed.

While the worker thread calculates the salary, the main application can perform other activities, such as:

- Displaying employee information
- Loading salary history
- Updating the user interface
- Processing another request

When the salary result is required, the application obtains it from the `Future`.

If the calculation is still running, the application waits until the result becomes available.

Example Flow

1. The employee requests the monthly salary details.
2. The application creates a salary calculation task.
3. The task is submitted to the executor.
4. The executor assigns the task to a worker thread.
5. The main application continues with other activities.
6. The worker thread calculates the final salary.
7. The salary amount is returned through the `Future`.
8. The application displays the salary to the employee.

Expected Result

The employee receives the calculated net salary without blocking the main application during the entire calculation process.

Key Learning

- `Callable` is used because the salary calculation returns a value.
- `Future` stores the result of the asynchronous task.
- `ExecutorService` manages the execution of the task.
- `Future.get()` is used when the calculated result is required.
- Exceptions occurring during salary calculation can be passed back to the main application.